ch1

August 13, 2026

1 Chapter 1 - Machine Learning and Scoring

A clinic measures the body mass index of 442 patients and wants to know how far their diabetes will have progressed a year from now. Nobody knows the formula. **Figure 1.1** shows a computer finding it from the measurements alone, its error falling every step.

Author: Dane Brown

Figure 1.1 shows a line learning. **Green dots** are 442 real patients: higher body mass index to the right, worse disease progression a year later upward. The **red line** is the **model's guess** at the relationship known as the Hypothesis. It starts highly incorrect, and each step it nudges its slope and intercept downhill, so the error printed in the title falls.

Nobody told the line where to sit. It found its own way there from the data. That is the whole of Section 1.5, before you have read a word of theory.

[1]:
```
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.datasets import load_diabetes
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML

plt.rcParams['animation.embed_limit'] = 80.0

diabetes_preview = load_diabetes()
xr = diabetes_preview.data[:, 2]
yr = diabetes_preview.target
xs_c = (xr - xr.mean()) / xr.std()

m, c, lr = -80.0, 40.0, 0.25
frames_mc = []
for _ in range(14):
    frames_mc.append((m, c))
    err = (m * xs_c + c) - yr
    m -= lr * (xs_c * err).mean()
    c -= lr * err.mean()
```

```
fig, ax = plt.subplots(figsize=(7, 4.8))
ax.scatter(xr, yr, s=14, c="#55A868", alpha=0.6)
ax.set_xlim(xr.min() - 0.02, xr.max() + 0.02); ax.set_ylim(0, 370)
ax.set_xlabel("body mass index (standardised units)"); ax.set_ylabel("disease␣
 ↪progression after one year")
line_fit, = ax.plot([], [], c="#C44E52", lw=2.5)
xs_plot = np.linspace(xr.min(), xr.max(), 50)
xs_plot_c = (xs_plot - xr.mean()) / xr.std()

def update(i):
    mm, cc = frames_mc[i]
    line_fit.set_data(xs_plot, mm * xs_plot_c + cc)
    mse = (((mm * xs_c + cc) - yr) ** 2).mean()
    ax.set_title(f"step {i}    average squared error = {mse:,.0f}".replace(",",␣
 ↪" "))
    return line_fit,

fig.text(0.5, 0.02, "Figure 1.1. A line learns to fit disease progression",␣
 ↪ha="center", fontsize=9)
plt.subplots_adjust(bottom=0.18)

anim0 = FuncAnimation(fig, update, frames=len(frames_mc), interval=900,␣
 ↪blit=False, repeat=True)
plt.close(fig)
import base64, tempfile, os
_g = tempfile.NamedTemporaryFile(suffix='.gif', delete=False); _g.close()
anim0.save(_g.name, writer='pillow')
with open(_g.name, 'rb') as _f: _d = _f.read()
os.unlink(_g.name)
HTML(f'<img src="{base64.b64encode(_d).decode()}">')
```

[1]: <IPython.core.display.HTML object>

1.0.1 What is this figure trying to demonstrate?

That fitting is a search, not a formula lookup. The red line starts in a position nobody would choose and no equation is solved anywhere in the animation. Each frame changes m and c a little in the direction that lowers the printed error, and after 14 such changes the line sits where a human would have drawn it. Section 1.5 names that procedure.

1.1 Lecture map

Section 1.1 What learning actually means, and the five moves every model in this material makes.

- **1.1.1** Three flavours of learning.
- **1.1.2** The cvML methodology, five steps.

Section 1.2 Scoring, because you cannot improve what you cannot grade.

- **1.2.1** The confusion matrix, four counts.
- **1.2.2** Accuracy, precision, recall and F1.
- **1.2.3** A classifier that scores well and is useless.

Section 1.3 k-Nearest Neighbours, classify by asking the neighbours.

- **1.3.1** One newcomer, three neighbours.
- **1.3.2** Choosing k, a bias-variance knob.
- **1.3.3** Splitting, fitting and scoring in a loop.

Section 1.4 Linear regression, fit a straight line and score it.

- **1.4.1** Body mass index against disease progression.
- **1.4.2** R-squared, the regression score.
- **1.4.3** Holding data back, and the golden rule.
- **1.4.4** The bias-variance dial, drawn.
- **1.4.5** Loss against metric, two different numbers.

Section 1.5 Gradient descent, the golden thread, one rule that rolls every model downhill.

Section 1.6 Where this goes next, plus **when it breaks**

1.2 Notation

Across these notebooks, m is the number of training samples and n is the number of features. A **dataset** is an $m \times n$ grid called `X`, one row per sample and one column per feature, plus a column of m labels called `y`. A model writes its **prediction** as $\hat{y}$, read "y-hat", to keep it apart from the true y.

Symbols used throughout.

- m: number of training samples
- n: number of features (dimensionality)
- X: data matrix, shape $m \times n$
- y: true labels (y_{true})
- $\hat{y}$: predictions (y_{pred}, the estimator's guess)

1.3 1.1. What is machine learning?

> Before you read on: in one sentence, what does it mean for a machine to *learn*? Write your answer down, then compare it to the definition below.

Tom Mitchell's 1997 definition is the one worth memorising.

> A computer program is said to **learn from experience E** with respect to some **task T** and **performance measure P**, if its performance at tasks in T, as measured by P, improves with experience E.

Three letters. **T** is the job. **E** is the data. **P** is the score. If more data pushes the score up, the machine is learning. That is the entire test.

1.4 Quick Recall

- **Learning** = performance at task **T**, measured by **P**, improves with experience **E**.
- **Supervised** learning has labels. **Unsupervised** has none. **Reinforcement** has rewards.
- **Classification** predicts a discrete label, **regression** predicts a continuous number.
- The scikit-learn API is four calls: `SomeModel(...)`, `.fit`, `.predict`, `.score`.

1.4.1 1.1.1 Three flavours of learning

Supervised learning. You hand the machine labelled examples, a photo *and* the word "cat", and it learns the mapping from one to the other. **Classification** predicts a discrete label, **regression** predicts a continuous number. Sections 1.2 to 1.5 of this chapter are supervised.

Unsupervised learning. You hand it unlabelled data and ask what structure is in there. No answer key exists. k-means in `ch3` groups points into clusters nobody labelled.

Reinforcement learning. No labels and no fixed dataset. An agent acts, the world responds with a reward or a penalty, and the agent learns which actions pay off over time. Game-playing programs and robot controllers learn this way.

1.4.2 1.1.2 The cvML methodology, five steps

Every model in this material is built with the same five moves. Memorise the verbs.

1. **Initialise** the model, pick the algorithm.
2. **Set parameters**, the numbers the model cannot learn for itself, like how many neighbours to ask.
3. **Fit**, feed it the training data so it learns.
4. **Predict**, ask the fitted model for guesses on data it has not seen.
5. **Score**, compare those guesses to the truth.

In scikit-learn those five verbs shrink into four method calls.

```
model = SomeModel(...)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
model.score(X_test, y_test)
```

That is the core of the API. Every algorithm in scikit-learn, from k-Nearest Neighbours to a neural network, plugs into those four calls, so learning a new model means learning what it does between `fit` and `predict`, never a new interface.

[2]:
```
from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(n_neighbors=3)
print(model)
print("fit ->", type(model.fit).__name__)
print("predict ->", type(model.predict).__name__)
print("score ->", type(model.score).__name__)
```

```
KNeighborsClassifier(n_neighbors=3)
fit -> method
```

```
predict -> method
score -> method
```

1.4.3 Quick check, Section 1.1

1. Name **T**, **E** and **P** for a spam filter.
2. Predicting house price from floor area is which of classification or regression, and why?
3. You have 5 000 photos and no labels. Which flavour of learning is available to you?

Answers

1. T is labelling an email spam or not, E is the archive of emails already labelled, P is the fraction labelled correctly.
2. Regression. The answer is a continuous number, not one of a fixed set of labels.
3. Unsupervised. With no answer key you can only ask what structure the photos already have.

1.5 1.2. Scoring: how good are the guesses?

Before any model is trained, the guesses need a way to be graded. The simplest grading tool is the **confusion matrix**, a 2-by-2 grid that counts hits and misses.

1.5.1 1.2.1 Simplest example, five animals guessed by hand

A toy classifier, "is it a cat?", looks at five photos and guesses. **The class which is positive is the Scikit's choice by default.** scikit-learn sorts the class labels it finds and treats the **last** one in sorted order as positive. With the integers 0 and 1 that sorts to `[0, 1]`, so 1 is positive: cats are positive because cat is coded 1, not because cats are more interesting.

Section 1.2.1a shows what happens when the labels are the strings `"cat"` and `"dog"` instead. The truth and the guesses live in two arrays, and the table below is printed from those arrays rather than typed out, so the numbers cannot drift apart.

1.6 Quick Recall

- The **confusion matrix** counts four things: TN, FP, FN, TP.
- **Accuracy** = (TP + TN) / everything. It lies whenever one class is rare.
- **Precision** = TP / (TP + FP), of the ones you called positive, how many were.
- **Recall** = TP / (TP + FN), of the ones that were positive, how many you caught.
- **F1** is the harmonic mean of precision and recall, so both must be high.
- scikit-learn treats the **last label in sorted order** as positive, and `pos_label` defaults to 1.

[3]:
```
import numpy as np
from IPython.display import Markdown, display

y_true = [1, 1, 0, 0, 0]
y_pred = [1, 0, 1, 0, 0]
names = {1: "cat", 0: "dog"}

rows = []
for k in range(len(y_true)):
```

```
    rows.append(f"| photo {k + 1} | {names[y_true[k]]} | {names[y_pred[k]]} |")
table_md = "| Photo | Truth | Guess |\n|---|---|---|\n" + "\n".join(rows)
display(Markdown(table_md))

wrong = [k + 1 for k, (t, g) in enumerate(zip(y_true, y_pred)) if t != g]
print("photos the classifier got wrong:", wrong)
```

Photo	Truth	Guess
photo 1	cat	cat
photo 2	cat	dog
photo 3	dog	cat
photo 4	dog	dog
photo 5	dog	dog

```
photos the classifier got wrong: [2, 3]
```

Count the four cells by hand first, with no metrics library. `(y_true == 1)` is a boolean array, and multiplying two boolean arrays is a logical AND, so each product is 1 only where both conditions hold. Summing counts them.

> Look at the five photos in the table above before you run the next cell. A **false positive** is a photo the classifier *called* a cat that is not one. Which photo number is it? And which photo is the **false negative**, a real cat called a dog? Say both out loud, then run the cell and check.

[4]:
```
y_true_arr = np.array(y_true)
y_pred_arr = np.array(y_pred)

true_negative  = np.sum((y_pred_arr == 0) * (y_true_arr == 0))
false_positive = np.sum((y_pred_arr == 1) * (y_true_arr == 0))
false_negative = np.sum((y_pred_arr == 0) * (y_true_arr == 1))
true_positive  = np.sum((y_pred_arr == 1) * (y_true_arr == 1))

print(f"TN = {true_negative}, FP = {false_positive}, "
      f"FN = {false_negative}, TP = {true_positive}")
```

```
TN = 2, FP = 1, FN = 1, TP = 1
```

[5]:
```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_true, y_pred)
print(cm)

tn, fp, fn, tp = cm.ravel()
print(f"True Negatives  (dog called dog): {tn}")
print(f"False Positives (dog called cat): {fp}")
print(f"False Negatives (cat called dog): {fn}")
print(f"True Positives  (cat called cat): {tp}")
```

```
[[2 1]
 [1 1]]
True Negatives  (dog called dog): 2
False Positives (dog called cat): 1
False Negatives (cat called dog): 1
True Positives  (cat called cat): 1
```

1.6.1 End of simplest example

Four counts, computed twice and agreeing: once by multiplying boolean arrays, once by `confusion_matrix`. Every classification metric in this course is a ratio of those four numbers.

1.6.2 1.2.1a The same five photos, labelled with words

Nothing above told scikit-learn that cat was positive. It sorted the labels it was given and took the last one. Swap the integers for the strings `"cat"` and `"dog"` and the sort order changes: `sorted(["cat", "dog"])` is `["cat", "dog"]`, so **dog** becomes the positive class and every count swaps places. The data is identical. Only the spelling of the labels changed.

Run the same five photos through the words `"cat"` and `"dog"`. The third print forces cat back to the positive class by naming the label order yourself.

[6]:
```
y_true_words = ["cat", "cat", "dog", "dog", "dog"]
y_pred_words = ["cat", "dog", "cat", "dog", "dog"]

print("sorted labels:", sorted(set(y_true_words)), "-> positive class is",␣
 ↪sorted(set(y_true_words))[-1])
print(confusion_matrix(y_true_words, y_pred_words))

print(confusion_matrix(y_true_words, y_pred_words, labels=["dog", "cat"]))
```

```
sorted labels: ['cat', 'dog'] -> positive class is dog
[[1 1]
 [1 2]]
[[2 1]
 [1 1]]
```

The first matrix counts with dog as positive, the second with cat as positive, from the same predictions. Two lessons, and the second is the one that bites in an assignment.

- `pos_label` defaults to 1, which is why `precision_score` and `recall_score` below score cats.
- If your labels are words, pass `labels=[...]` explicitly, or your precision and recall silently describe the other class.

1.6.3 1.2.2 Four metrics, all built from four counts

Table 1.1. Four metrics, each built from one quadrant of the confusion grid

Those four letters - **TN, FP, FN, TP** - are the atoms of every classification metric. Memorise their positions in the grid; the next four metrics are just ratios of them.

Name	Asks	Formula
Accuracy	out of every prediction, how many were right?	$\frac{TP+TN}{TP+TN+FP+FN}$
Precision	of the ones **called** positive, how many really were?	$\frac{TP}{\underbrace{TP+FP}_{\text{guesses called positive}}}$
Recall	of the **real** positives, how many were caught?	$\frac{TP}{\underbrace{TP+FN}_{\text{actual positives}}}$
F1	balance of precision and recall (their harmonic mean)	$\frac{2 \cdot P \cdot R}{P+R}$

Two of those denominators are scaffolded with *underbrace* so you can *read* them in English: "precision divides the true positives by everything labelled positive".

[7]:
```
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score)

accuracy  = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred)
recall    = recall_score(y_true, y_pred)
f1        = f1_score(y_true, y_pred)

print(f"accuracy  = {accuracy:.2f}")
print(f"precision = {precision:.2f}")
print(f"recall    = {recall:.2f}")
print(f"f1        = {f1:.2f}")
```

```
accuracy  = 0.60
precision = 0.50
recall    = 0.50
f1        = 0.50
```

1.6.4 1.2.3 A classifier that scores 92% and is worthless

Four metrics is three more than most people bother with, and the reason is easiest to see by cheating.

Here is the dumbest possible model. It ignores the data completely and always answers "not spam". Run it on a mailbox where 92 out of 100 messages really are not spam.

> Before you run the cell: what accuracy do you think a model that never looks at its input can reach? Write a number down.

[8]:
```
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score
```

```python
rng = np.random.default_rng(0)
n = 100
y_mail = np.zeros(n, dtype=int)
y_mail[:8] = 1
rng.shuffle(y_mail)

X_mail = rng.normal(size=(n, 3))

majority = DummyClassifier(strategy="most_frequent").fit(X_mail, y_mail)
coin = DummyClassifier(strategy="uniform", random_state=0).fit(X_mail, y_mail)

print("real spam in the mailbox:", y_mail.sum(), "of", n)
print()
print("majority-guess (always predict not-spam):")
print("  accuracy =", round(accuracy_score(y_mail, majority.predict(X_mail)),␣
 ↪2),
      " recall =", round(recall_score(y_mail, majority.predict(X_mail),␣
 ↪zero_division=0), 2),
      " precision =", round(precision_score(y_mail, majority.predict(X_mail),␣
 ↪zero_division=0), 2))
print("coin-flip    (guess spam half the time):")
print("  accuracy =", round(accuracy_score(y_mail, coin.predict(X_mail)), 2),
      " recall =", round(recall_score(y_mail, coin.predict(X_mail),␣
 ↪zero_division=0), 2))
print()
print("On balanced data a coin flip scores about 0.50, that is the floor.")
print("On this imbalanced mailbox the majority-guess cheats its way to 0.92␣
 ↪without reading a single message.")
```

```
real spam in the mailbox: 8 of 100

majority-guess (always predict not-spam):
  accuracy = 0.92  recall = 0.0  precision = 0.0
coin-flip    (guess spam half the time):
  accuracy = 0.46  recall = 0.62

On balanced data a coin flip scores about 0.50, that is the floor.
On this imbalanced mailbox the majority-guess cheats its way to 0.92 without
reading a single message.
```

On balanced data (equal positives and negatives) a coin flip scores **50% accuracy**, no better and no worse than guessing. That 50% is the floor: any real model has to beat it before it has learned anything the coin did not already know for free. On imbalanced data the majority-class guess is an even higher floor, and it can look respectable, which is exactly the trap the mailbox demo above walks into.

> The majority-guess model is 92% accurate. Before you open this, write down what its recall is. It never flagged a single spam message.

Show what those three numbers mean

92% accurate, catches nothing. The model never flagged a single spam message, so recall is 0. Every real spam message slipped through, and accuracy did not notice, because accuracy counts the 92 easy correct answers and the 8 failures with equal weight.

This is **class imbalance**, and it is the normal state of affairs. Fraud, disease, defects and spam are all rare by definition, so the majority-class answer is always available and always looks good.

Now flip the cheat around. A model that flags *everything* as spam catches all 8 real spam messages, so its recall is a perfect 1.0.

> What has that perfect recall cost? Give precision a number out of the same 100 messages before you open the answer.

Show the cost of the second cheat

Precision falls to **0.08**. The model flagged all 100 messages, only 8 of them were really spam, and 8 out of 100 is 0.08. Every innocent message went in the bin to buy that perfect recall.

Neither cheat survives looking at precision and recall together. That is why the confusion matrix has four cells and not one, and why a single number like accuracy is never enough on its own.

> Which of the two cheats would you rather deploy on a cancer screen, and which on a spam filter? The answer is not the same, and that asymmetry is the entire reason you choose a metric before you train.

> **The golden rule of baselines.** A model that never looks at the input is the floor. On balanced classes that floor is a coin flip at 50%. On imbalanced classes the floor is the majority guess, and it can look impressive. Any model that does not clear its floor has learned nothing the floor did not already know for free.

The confusion grid and the four scores. **Figure 1.3** is the figure to stare at, because every score in every later chapter is read off a grid like it.

Figure 1.2 just below is the static version, four bars already drawn. **Figure 1.3**, the cell after that, animates the build.

Left, the confusion grid. Rows are the truth, columns are the guess. The diagonal running top-left to bottom-right holds the answers the model got right, darker blue where the count is higher. Anything off that diagonal is a mistake, and *which* mistake matters: top-right is a dog called a cat, bottom-left is a cat called a dog.

Right, four bars. Every bar is computed from those same four numbers and nothing else. No new information enters between the left panel and the right one, which is why arguing about metrics is really arguing about which of the four cells you care about.

Figure 1.3 lights up the four cells one at a time, then raises the bars, so you can see each score being built out of the counts it came from.

[9]:
```
import matplotlib.pyplot as plt

metrics = {"accuracy": accuracy, "precision": precision, "recall": recall, "f1":
  ↪ f1}
```

```
fig = plt.figure(figsize=(6.5, 4.2))
plt.bar(metrics.keys(), metrics.values(), color="#4C72B0")
for i, v in enumerate(metrics.values()):
    plt.text(i, v + 0.02, f"{v:.2f}", ha="center")
plt.ylim(0, 1.1); plt.ylabel("score (0 to 1)")
fig.text(0.5, 0.01, "Figure 1.2. Four metrics from the same counts",␣
 ↪ha="center", fontsize=10)
plt.tight_layout(); plt.show()
```

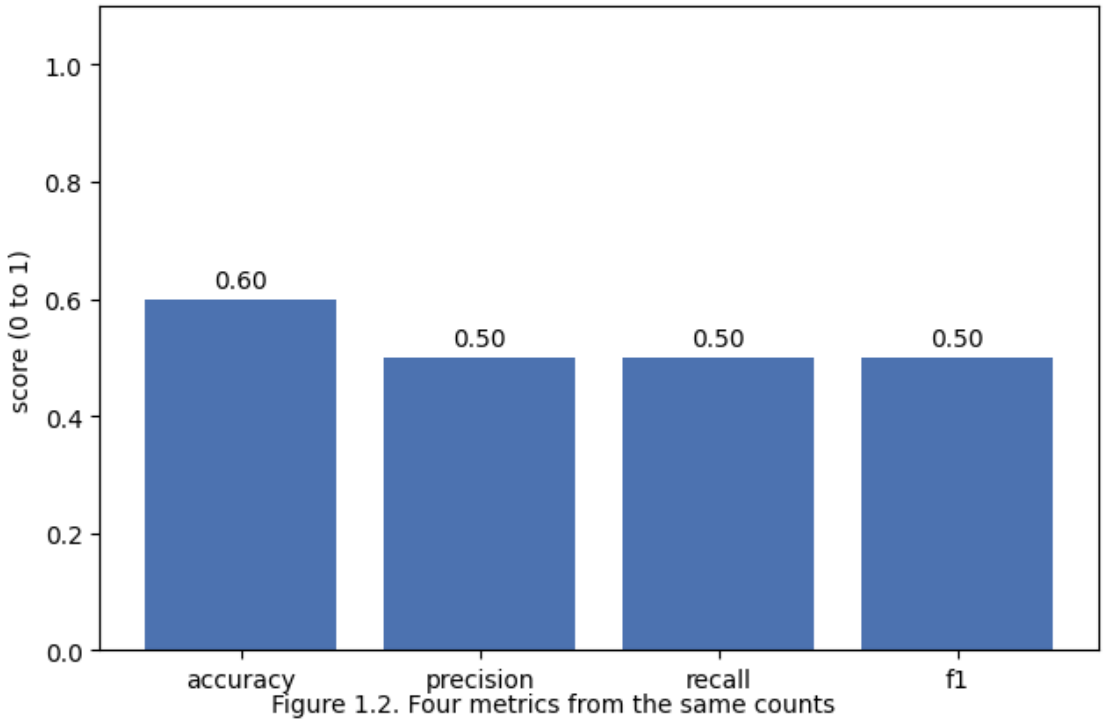


Figure 1.2. Four metrics from the same counts

[10]:
```
plt.rcParams['animation.embed_limit'] = 80.0
from matplotlib import animation as anim
from IPython.display import HTML
import base64, tempfile, os

fig, axes = plt.subplots(1, 2, figsize=(11, 4.6))
axL, axR = axes

grid = np.array([[tn, fp], [fn, tp]])
labels = [["TN", "FP"], ["FN", "TP"]]
axL.imshow(grid, cmap="Blues")
for (r, c), v in np.ndenumerate(grid):
    axL.text(c, r, f"{labels[r][c]}\n{v}", ha="center", va="center",
             fontsize=14, color="black" if v > 0 else "grey")
```

```
axL.set_xticks([0, 1]); axL.set_xticklabels(["dog (0)", "cat (1)"])
axL.set_yticks([0, 1]); axL.set_yticklabels(["dog (0)", "cat (1)"])
axL.set_xlabel("Guess"); axL.set_ylabel("Truth")
axL.set_title("Confusion matrix (cat = positive)")

metric_names = ["accuracy", "precision", "recall", "f1"]
metric_vals = [accuracy, precision, recall, f1]
bars = axR.bar(metric_names, [0, 0, 0, 0], color="#4C72B0")
axR.set_ylim(0, 1.1); axR.set_ylabel("score (0 to 1)")
axR.set_title("Four metrics from the same counts")
bar_txts = [axR.text(i, 0.02, "", ha="center") for i in range(4)]

reveal_order = [(0, 0, "TN"), (0, 1, "FP"), (1, 0, "FN"), (1, 1, "TP")]

def update(i):
    if i < 4:
        r, c, name = reveal_order[i]
        axL.add_patch(plt.Rectangle((c - 0.5, r - 0.5), 1, 1,
                      fill=False, edgecolor="#C44E52", lw=3))
        axL.set_title(f"Reveal {name} = {grid[r, c]}")
    else:
        j = i - 4
        for k in range(j + 1):
            bars[k].set_height(metric_vals[k])
            bar_txts[k].set_text(f"{metric_vals[k]:.2f}")
            bar_txts[k].set_position((k, metric_vals[k] + 0.02))
        axL.set_title("Confusion matrix (cat = positive)")

anim_obj = anim.FuncAnimation(fig, update, frames=8, interval=900, blit=False,␣
 ↪repeat=True)
fig.text(0.5, 0.02, "Figure 1.3. Confusion grid and four scores", ha="center",␣
 ↪fontsize=9)
plt.subplots_adjust(bottom=0.18)
plt.close(fig)
_g = tempfile.NamedTemporaryFile(suffix='.gif', delete=False); _g.close()
anim_obj.save(_g.name, writer='pillow')
with open(_g.name, 'rb') as _f: _d = _f.read()
os.unlink(_g.name)
HTML(f'<img src="{base64.b64encode(_d).decode()}">')
```

[10]: <IPython.core.display.HTML object>

1.6.5 What is this figure trying to demonstrate?

That the four scores are four different questions asked of the same four counts. Nothing new enters the figure when a metric is revealed: each one is a ratio built from TN, FP, FN and TP. Watch which cells light up for precision against recall, because that difference is the whole reason a model can score 0.99 on one and 0.20 on the other.

1.6.6 Quick quiz

1. If a model predicts **every** photo is a cat, what happens to precision and recall? (Which one is perfect, which one falls?)
2. A dataset has 100 healthy patients and 10 sick ones. The model predicts "healthy" for everyone. What is its **accuracy**? Why is accuracy a misleading score here?
3. Of the four metrics above, which one answers "if the model says I am sick, should I worry?"
4. Two classifiers get the same accuracy but different F1. Which metric should you trust to pick between them, and why?

Answers (click reveal)

1. **Recall becomes 1.0** (every real cat is caught, because everything is called a cat), but **precision falls** (most of the cat guesses are actually dogs). Calling everything positive is the recall-maximising, precision-destroying strategy.
2. Accuracy $= \frac{100}{110} \approx 0.91$. It looks great (91%) but the model is useless - it never catches a single sick patient. When classes are imbalanced, accuracy hides failure.
3. **Precision**. "If the model says I am sick" = "of the positive guesses, how many are real?" That is the precision question.
4. **F1**. Same accuracy can hide very different precision/recall trade-offs; F1 blends the two so it punishes a model that is lopsided.

1.6.7 Quick check, Section 1.2

1. A test flags 1 patient in 1 000 and the disease affects 1 in 1 000. A model that always says "healthy" scores what accuracy?
2. Which of precision and recall does a cancer screening programme care about more, and why?
3. Your labels are `"benign"` and `"malignant"`. Without passing `labels=`, which one does `precision_score` treat as positive?

Answers

1. 99.9%, while catching nobody. Accuracy on an imbalanced problem is close to meaningless.
2. Recall. A missed cancer costs far more than a false alarm that a second test clears.
3. `"malignant"`, because it sorts after `"benign"` and scikit-learn takes the last sorted label.

1.7 1.3. k-Nearest Neighbours: classify by asking the neighbours

The first real learning algorithm is simple, and the best way in is a stadium.

Picture a rugby match. The home supporters fill one stand, the away supporters fill another. A newcomer walks in late and stands somewhere in the middle of the field, not sure which stand to join. The simplest possible rule: look at the **k** supporters nearest to you on the field, see which team most of them support, and join that stand.

That is **k-Nearest Neighbours**, and it is the whole algorithm:

1. Look at its **k** nearest training points (nearest by plain distance).
2. Take a **majority vote** of their labels.
3. That majority is your prediction.

No maths to fit, no parameters to learn. The training data *is* the model. `k` is the only knob.

1.7.1 1.3.1 One newcomer, three neighbours

Thirty supporters are scattered on the field while heading to the stands: blue squares (home) cluster bottom-left, red triangles (away) cluster top-right. The green circle is the newcomer. **Figure 1.6** asks 3-NN which stand the newcomer joins, with lime lines from the newcomer to each ringed neighbour making the vote visible.

1.8 Quick Recall

- k-NN predicts by **majority vote** of the `k` nearest training points. There is no fitting step.
- The training data **is** the model, so prediction cost grows with the size of the training set.
- Small `k` follows every point: **low bias, high variance**, an overfit boundary.
- Large `k` averages across classes: **high bias, low variance**, an underfit boundary.
- `k` is chosen on validation data, never on the test set.

1.8.1 1.3.1a Simplest example: one supporter, one coordinate

A supporter standing on the field is two numbers, coordinates, with a position across and a position up, drawn on a cartesian plane.

Two seed mechanisms appear in this notebook. `np.random.seed(42)` (next cell) sets the global NumPy state, so every `np.random.*` call after it is reproducible. `random_state=42`, used by scikit-learn from Section 1.3.2 onward, seeds a single function call. Same number, two mechanisms: one for NumPy, one for scikit-learn. Remember several algorithms are stochastic in nature since they start with a random guess (simple hypothesis being $y = mx + c$).

[11]:
```
np.random.seed(42)

single_supporter = np.random.randint(0, 100, 2)
single_label = np.random.randint(0, 2)

print("one supporter, two features (across, up):", single_supporter)
print("their label, 0 = home, 1 = away:", single_label)
```

```
one supporter, two features (across, up): [51 92]
their label, 0 = home, 1 = away: 0
```

1.8.2 1.3.1b One supporter becomes many: `generate_data`

Wrap that in a function and the number of supporters, m, and the number of features, n, become arguments. Everything downstream reads `train_data` and `train_labels`, so the rest of the notebook does not care how they were made.

Note the shapes. `train_data` is $m \times n$, one row per supporter. `train_labels` is a column of m values, which scikit-learn wants flattened to one dimension with `ravel()`.

[12]:
```
def generate_data(num_samples, num_features):
    """num_samples supporters, each with num_features coordinates, plus a 0/1␣
 ↪label."""
    data_size = (num_samples, num_features)
    data = np.random.randint(0, 100, size=data_size)
```

```
    labels_size = (num_samples, 1)
    labels = np.random.randint(0, 2, size=labels_size)
    return data.astype(np.float32), labels

m_samples, n_features = 11, 2
train_data, train_labels = generate_data(m_samples, n_features)

print("train_data shape:", train_data.shape, " (m samples, n features)")
print("train_labels shape:", train_labels.shape, "-> flattened:", train_labels.
 ↪ravel().shape)
print(train_data)
print(train_labels.ravel())
```

```
train_data shape: (11, 2)  (m samples, n features)
train_labels shape: (11, 1) -> flattened: (11,)
[[71. 60.]
 [20. 82.]
 [86. 74.]
 [74. 87.]
 [99. 23.]
 [ 2. 21.]
 [52.  1.]
 [87. 29.]
 [37.  1.]
 [63. 59.]
 [20. 32.]]
[1 1 1 0 1 0 0 0 0 0 1]
```

1.8.3 1.3.1c The function that draws the coordinate plane: `plot_data`

Splitting the supporters by label is one boolean mask each, the same mask from `ch0` section 0.2.4. `plot_data` then draws one marker per supporter on a plain coordinate plane. Every k-NN figure below is this function plus one extra point.

Figure 1.4 draws eleven supporters on the coordinate plane. Home supporters sit as blue squares, away supporters as red triangles. Every k-NN figure after this reuses the same drawing function with one extra point added.

Figure 1.5 adds one newcomer to the same field. The green dot lands among the blue and red supporters. The question k-NN answers is simple: which camp votes on the newcomer?

[13]:
```
def plot_data(all_blue, all_red):
    """Draw two labelled groups of supporters on a coordinate plane."""
    fig = plt.figure(figsize=(6.4, 5.0))
    plt.scatter(all_blue[:, 0], all_blue[:, 1], c="#4C72B0", marker="s", s=90,
                label="home supporters (label 0)")
    plt.scatter(all_red[:, 0], all_red[:, 1], c="#C44E52", marker="^", s=90,
                label="away supporters (label 1)")
    plt.xlabel("position across the field (feature 1)")
```

```
    plt.ylabel("position up the field (feature 2)")
    plt.legend(loc="best")

blue = train_data[train_labels.ravel() == 0]
red = train_data[train_labels.ravel() == 1]

plot_data(blue, red)
fig = plt.gcf()
fig.text(0.5, 0.01, "Figure 1.4. Eleven supporters on the field", ha="center",␣
 ↪fontsize=10)
plt.show()
```

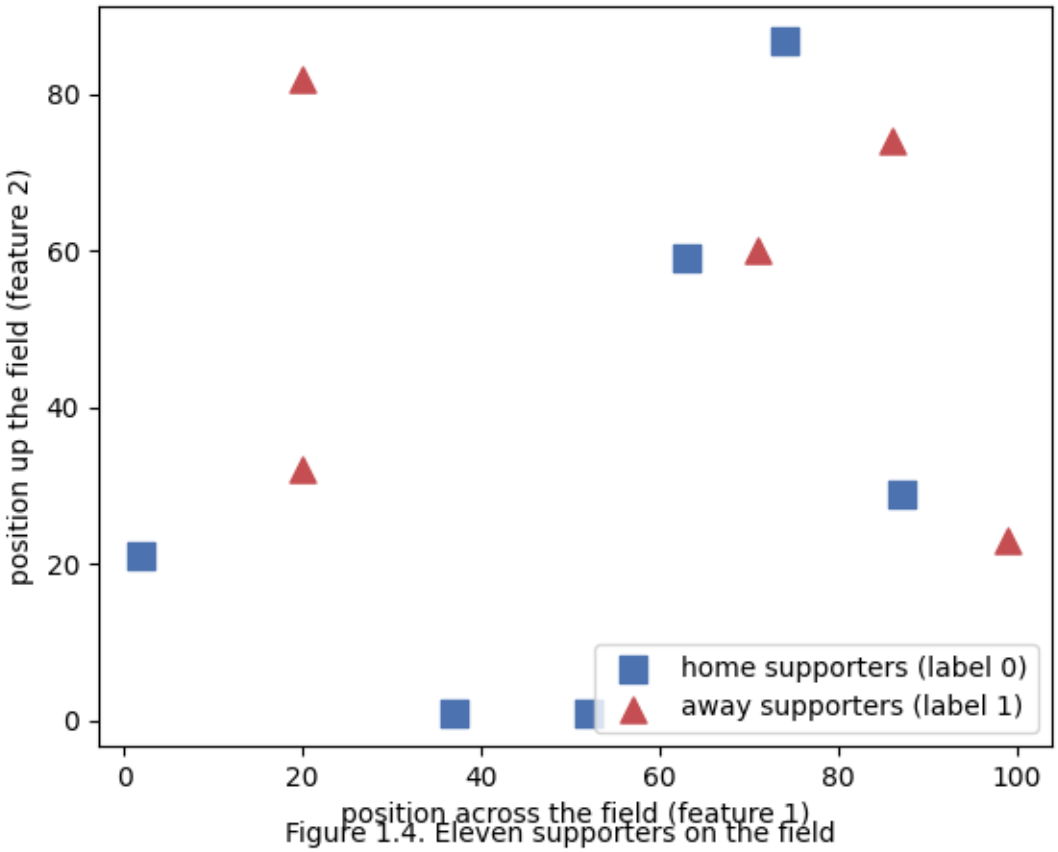


Figure 1.4. Eleven supporters on the field

[14]:
```
newcomer, _ = generate_data(1, 2)
plot_data(blue, red)
fig = plt.gcf()  # capture current figure for caption
plt.plot(newcomer[0, 0], newcomer[0, 1], "go", markersize=14)
fig.text(0.5, 0.01, "Figure 1.5. Eleven supporters and one newcomer",␣
 ↪ha="center", fontsize=10)
plt.show()
```

```
print("the newcomer stands at", newcomer[0])
```

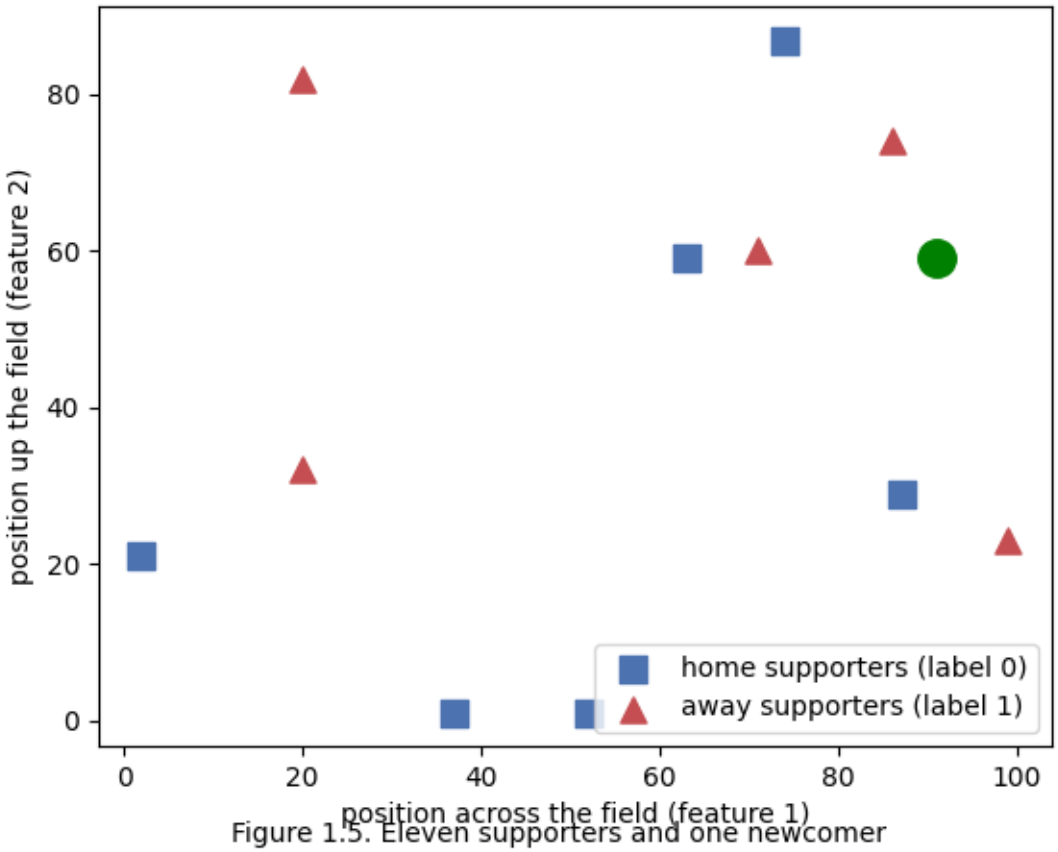


Figure 1.5. Eleven supporters and one newcomer

```
the newcomer stands at [91. 59.]
```

Look at where the green newcomer landed. Which stand would **you** put them in, and how many of the nearest supporters did you look at to decide? That number is `k`.

1.8.4 End of simplest example

Eleven supporters, one newcomer, no model yet. The next cell hands the same job to `KNeighborsClassifier`, which does exactly what you just did by eye.

[15]:
```
from sklearn.datasets import make_blobs
from sklearn.neighbors import KNeighborsClassifier

X, y = make_blobs(n_samples=30, centers=[(2, 2), (7, 7)],
                  cluster_std=1.1, random_state=42)
query = np.array([[4.5, 4.5]])

knn = KNeighborsClassifier(n_neighbors=3).fit(X, y)
```

```
stand = "home (blue)" if knn.predict(query)[0] == 0 else "away (red)"
print("3-NN says the newcomer joins the", stand, "stand")

fig = plt.figure(figsize=(6.8, 5.4))
plt.scatter(X[y == 0, 0], X[y == 0, 1], marker="s", s=90,
            c="#4C72B0", label="home supporters")
plt.scatter(X[y == 1, 0], X[y == 1, 1], marker="^", s=90,
            c="#C44E52", label="away supporters")
plt.scatter(query[:, 0], query[:, 1], marker="o", s=220,
            c="#55A868", edgecolors="black", label="newcomer", zorder=5)

dists = np.linalg.norm(X - query, axis=1)
nn_idx = np.argsort(dists)[:3]
plt.scatter(X[nn_idx, 0], X[nn_idx, 1], s=320, facecolors="none",
            edgecolors="lime", linewidths=2.5, zorder=4)
for p in X[nn_idx]:
    plt.plot([query[0, 0], p[0]], [query[0, 1], p[1]],
             "lime", alpha=0.5, lw=1.7, zorder=3)
plt.legend(loc="upper left"); plt.axis("equal")
plt.xlabel("position across the field"); plt.ylabel("position up the field")
fig.text(0.5, 0.01, "Figure 1.6. k-NN: one newcomer, three neighbours vote",␣
 ↪ha="center", fontsize=10)
plt.show()
```

```
3-NN says the newcomer joins the home (blue) stand
```

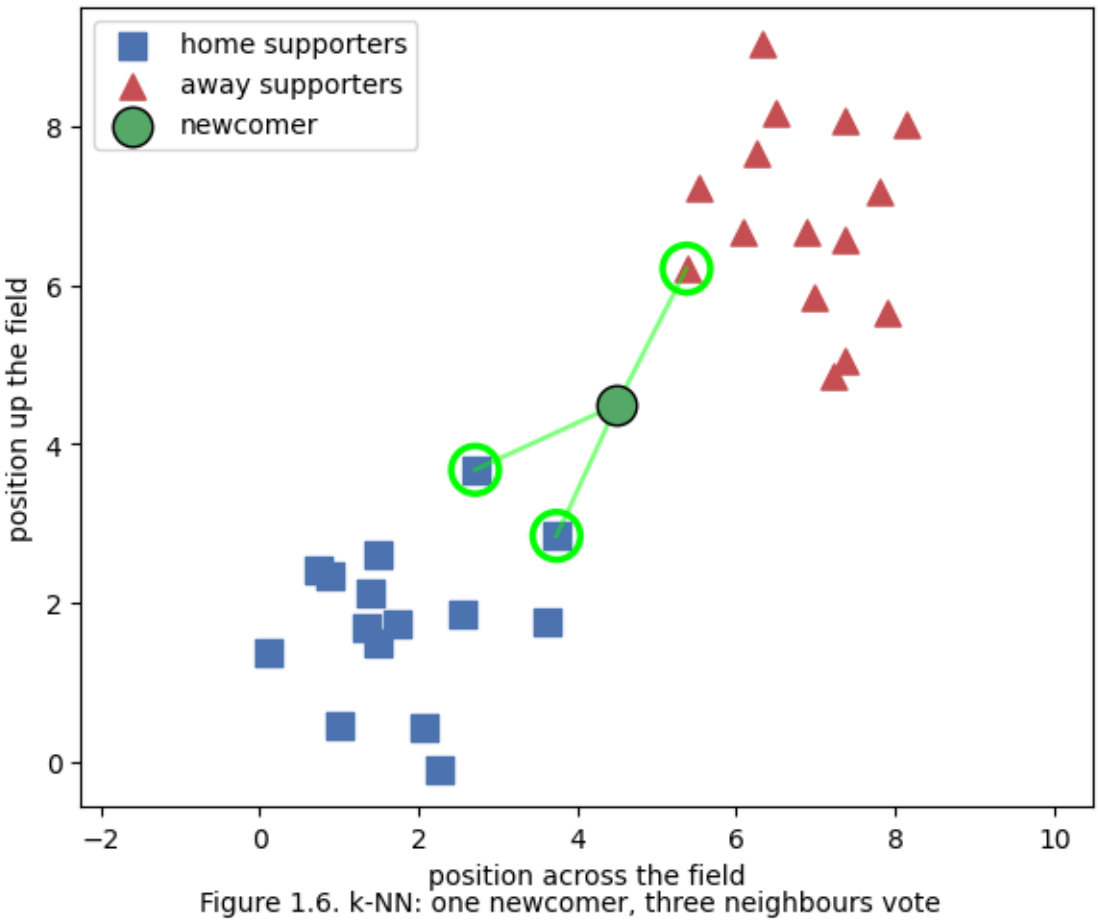


Figure 1.6. k-NN: one newcomer, three neighbours vote

1.8.5 1.3.2 Choosing k, a bias-variance knob

k is a dial between two failure modes:

- **k = 1** - the query just copies its single nearest neighbour. The boundary wriggles around every point. **High variance, overfit** (memorises noise).
- **k = large** - the query asks so many neighbours it ignores local structure. The boundary smooths into a boring line. **High bias, underfit** (too coarse).
- **Odd k** (3, 5, 7…) avoids ties in binary classification.

The value of k is not picked by eye. A grid search over candidate values, scored by cross-validation, keeps the best one. Figure 1.7 snapshots three fixed k values on the same dataset; Figure 1.8 sweeps k from 1 to 55.

[16]:
```
fig, axes = plt.subplots(1, 3, figsize=(13.5, 4.6))

Xn, yn = make_blobs(n_samples=60, centers=[(2, 2), (7, 7)],
                    cluster_std=2.0, random_state=42)
xx, yy = np.meshgrid(np.linspace(-3, 12, 120), np.linspace(-3, 12, 120))
```

```
grid = np.c_[xx.ravel(), yy.ravel()]

for ax, k in zip(axes, [1, 7, 25]):
    knn = KNeighborsClassifier(n_neighbors=k).fit(Xn, yn)
    Z = knn.predict(grid).reshape(xx.shape)
    ax.contourf(xx, yy, Z, alpha=0.25, cmap="coolwarm")
    ax.scatter(Xn[yn == 0, 0], Xn[yn == 0, 1], marker="s", c="#4C72B0", s=30)
    ax.scatter(Xn[yn == 1, 0], Xn[yn == 1, 1], marker="^", c="#C44E52", s=30)
    ax.set_title(f"k = {k}\n{'(wiggly: overfit)' if k == 1 else '(smooth: underfit)' if k == 25 else '(balanced)'}")
plt.subplots_adjust(top=0.82)
fig.text(0.5, 0.01, "Figure 1.7. Choosing k, the bias-variance dial", ha="center", fontsize=10)
plt.tight_layout(); plt.show()
```

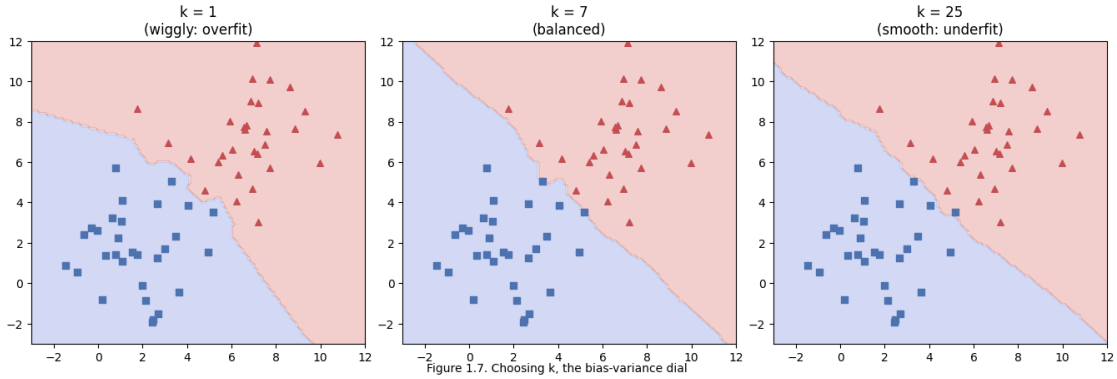


Figure 1.7. Choosing k, the bias-variance dial

[17]:
```
fig, ax = plt.subplots(figsize=(7, 5.8))
ax.scatter(Xn[yn == 0, 0], Xn[yn == 0, 1], marker="s", c="#4C72B0", s=30,
           label="class 0")
ax.scatter(Xn[yn == 1, 0], Xn[yn == 1, 1], marker="^", c="#C44E52", s=30,
           label="class 1")
ax.set_xlim(-3, 12); ax.set_ylim(-3, 12); ax.set_aspect("equal")
ax.legend(loc="upper left")

contours = []
k_seq = [1, 2, 3, 5, 7, 11, 15, 21, 29, 41, 55]

def update(i):
    k = min(k_seq[i], len(Xn) - 1)
    for cf in contours:
        cf.remove()
    contours.clear()
    knn = KNeighborsClassifier(n_neighbors=k).fit(Xn, yn)
    Z = knn.predict(grid).reshape(xx.shape)
```

```
    cf = ax.contourf(xx, yy, Z, alpha=0.25, cmap="coolwarm")
    contours.append(cf)
    label = "overfit (high variance)" if k <= 3 else ("underfit (high bias)" if␣
 ↪k >= 29 else "balanced")
    ax.set_title(f"k = {k} - {label}")
    return []

fig.text(0.5, 0.02, "Figure 1.8. k turns the bias-variance knob", ha="center",␣
 ↪fontsize=9)
plt.subplots_adjust(top=0.90, bottom=0.12)

anim_obj = anim.FuncAnimation(fig, update, frames=len(k_seq), interval=900,␣
 ↪blit=False, repeat=True)
plt.close(fig)
_g = tempfile.NamedTemporaryFile(suffix=".gif", delete=False); _g.close()
anim_obj.save(_g.name, writer="pillow")
with open(_g.name, "rb") as _f: _d = _f.read()
os.unlink(_g.name)
HTML(f'<img src="{base64.b64encode(_d).decode()}">')
```

[17]:
```
<IPython.core.display.HTML object>
```

1.8.6 What is this figure trying to demonstrate?

That `k` controls how much attention the boundary pays to individual points. At `k = 1` the boundary detours around every point it can see. As `k` grows the same boundary straightens, because each prediction is now an average over more neighbours. Nothing about the data changes across the frames. One number does.

1.8.7 1.3.2a A dataset the dial can actually be seen on

Two round clusters 5 units apart are separable by almost any k, so Figures 1.7 and 1.8 show the boundary changing shape without the score changing much. To watch overfitting cost you something, the classes have to interlock. `make_moons` with noise does that: two crescents that overlap along their facing edges.

Now the same dial moves both scores, and it moves them in opposite directions.

> With interlocking crescents, which do you expect for `k = 1`, a high or a low **training** score? And what about its **test** score? Write both down before running the cell.

Figure 1.9 shows two interlocking crescents where the choice of k costs you something. At small k the boundary wriggles into the gap. At large k it smooths over and misclassifies the overlap.

[18]:
```
from sklearn.datasets import make_moons
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split

Xm, ym = make_moons(n_samples=300, noise=0.30, random_state=42)
```

```
Xm_train, Xm_test, ym_train, ym_test = train_test_split(
    Xm, ym, test_size=0.3, random_state=42)

print(" k    training   test")
for k in [1, 5, 15, 51]:
    knn_m = KNeighborsClassifier(n_neighbors=k).fit(Xm_train, ym_train)
    print(f"{k:2d}     {knn_m.score(Xm_train, ym_train):.3f}      {knn_m.
 ↪score(Xm_test, ym_test):.3f}")

xxm, yym = np.meshgrid(np.linspace(Xm[:, 0].min() - 0.5, Xm[:, 0].max() + 0.5,␣
 ↪200),
                       np.linspace(Xm[:, 1].min() - 0.5, Xm[:, 1].max() + 0.5,␣
 ↪200))
grid_m = np.c_[xxm.ravel(), yym.ravel()]

fig, axes = plt.subplots(1, 3, figsize=(13.5, 4.4))
for ax, k in zip(axes, [1, 15, 51]):
    knn_m = KNeighborsClassifier(n_neighbors=k).fit(Xm_train, ym_train)
    ax.contourf(xxm, yym, knn_m.predict(grid_m).reshape(xxm.shape), alpha=0.25,␣
 ↪cmap="coolwarm")
    ax.scatter(Xm_train[ym_train == 0, 0], Xm_train[ym_train == 0, 1],␣
 ↪marker="s", c="#4C72B0", s=18)
    ax.scatter(Xm_train[ym_train == 1, 0], Xm_train[ym_train == 1, 1],␣
 ↪marker="^", c="#C44E52", s=18)
    ax.set_xticks([]); ax.set_yticks([])
    ax.set_title(f"k = {k}   train {knn_m.score(Xm_train, ym_train):.2f}   "
                 f"test {knn_m.score(Xm_test, ym_test):.2f}", fontsize=10)
fig.text(0.5, 0.01, "Figure 1.9. Interlocking classes, where k costs you␣
 ↪something", ha="center", fontsize=10)
plt.tight_layout(rect=[0, 0.04, 1, 1]); plt.show()
```

```
 k    training   test
 1     1.000      0.867
 5     0.914      0.922
15     0.886      0.911
51     0.881      0.900
```

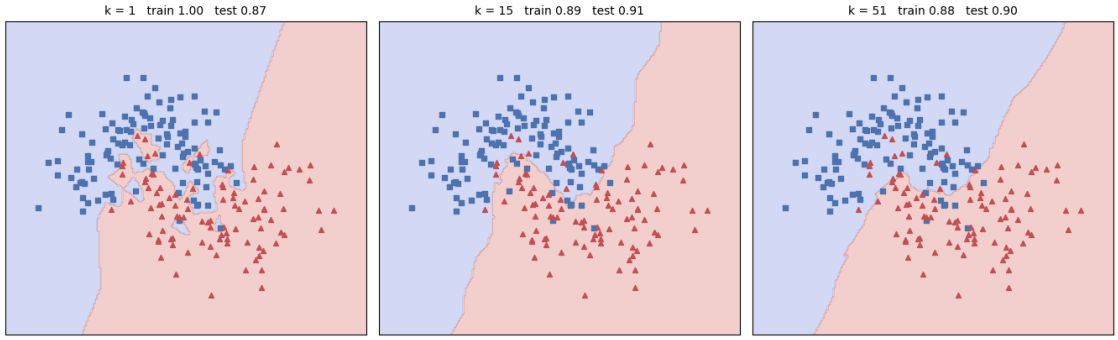


Figure 1.9. Interlocking classes, where k costs you something

`k = 1` scores 1.000 on the data it memorised and drops on data it has not seen: that gap is **variance**. `k = 51` scores lower on both, because the vote now reaches across the gap between the crescents and blurs them: that is **bias**. The best test score sits between the two, which is the whole reason `k` is chosen by cross-validation rather than by eye.

1.8.8 1.3.3 Splitting, fitting and scoring in a loop

So far `knn` was fitted on every available point, so nothing was left to test it on. The proper version splits the supporters into a training set and a held-back test set, fits on the first and scores on the second.

Wrapping that in a loop over `k` is the standard way to pick the knob: build, fit, score, keep the best.

> Thirty supporters, two well-separated clusters. Before you run it, write down whether you expect `k = 1` or `k = 9` to score higher on the held-back set.

[19]:
```
from sklearn.model_selection import train_test_split

Xk, yk = make_blobs(n_samples=200, centers=[(2, 2), (7, 7)],
                    cluster_std=2.0, random_state=42)
Xk_train, Xk_test, yk_train, yk_test = train_test_split(
    Xk, yk, test_size=0.3, random_state=42)

for k in [1, 3, 5, 9, 15]:
    model = KNeighborsClassifier(n_neighbors=k).fit(Xk_train, yk_train)
    print(f"k = {k:2d}   test accuracy = {model.score(Xk_test, yk_test):.3f}")
```

```
k =  1   test accuracy = 0.917
k =  3   test accuracy = 0.933
k =  5   test accuracy = 0.950
k =  9   test accuracy = 0.967
k = 15   test accuracy = 0.983
```

> **Forward pointer.** The bias-variance picture with a true curve and an irreducible-noise band, the one from the parent course, is **Figure 1.14** in Section 1.4.4a. It needs a regression setting before the noise band means anything, so it waits until Section 1.4.

1.8.9 Quick quiz

1. A new point sits *exactly* between a blue square and a red triangle. With **k = 2**, what goes wrong, and which simple fix (mentioned above) removes the problem?
2. You double the amount of training data. Does k-NN need to retrain in the `fit()` sense, or does it just store more points? What does that tell you about its "training time"?
3. k-NN relies on distance. If one feature is measured in kilometres and another in millimetres, what will go wrong? (Scaling, the fix, is the first topic of `ch2`.)
4. Why is k = 1 usually a bad idea on noisy data? Answer in terms of variance.

Answers (click reveal)

1. A tie - one vote each. The fix is to use an **odd k** for binary classification so a vote can never split 50/50.
2. k-NN has **no training step** in the usual sense; `fit()` just stores the data. Training "time" is ~zero, but prediction time grows with the dataset (it must compute distances to every stored point).
3. The millimetre feature dominates the distance (its numbers are ~a million times bigger). k-NN becomes "k-nearest by millimetres". This is why **feature scaling** is needed - the topic of `ch2`'s preprocessing section.
4. k = 1 copies whichever single point is nearest. On noisy data that nearest point is often a mislabelled outlier, so the prediction is noisy too - high variance. Larger k averages out the noise.

1.8.10 Quick check, Section 1.3

1. What is the training accuracy of a 1-NN classifier on its own training data, and why is the number useless?
2. Two classes, `k = 6`, and the vote comes out 3 against 3. What have you done wrong?
3. Why does k-NN take longer to predict than linear regression, given that it takes no time to train?

Answers

1. 100%. Every point is its own nearest neighbour, so the model is grading its own memory.
2. Chosen an even `k` for a two-class problem, which allows ties. Use an odd `k`.
3. Every prediction measures the distance to all m training points. Linear regression only evaluates $mx + c$.

1.9 1.4. Linear regression: fit a straight line

Classification guessed a label. **Regression** guesses a number. The simplest regression model draws a straight line through the data. A line has two knobs: a slope and an intercept.

Show the equation of a straight line

$$\hat{y} = m \cdot x + c$$

Here m is the slope (how steeply y rises per unit of x) and c is the intercept (where the line crosses the y-axis). The model "learns" by picking the m and c that make the line pass as close as possible to all the training points. It minimises the total squared error between the line and the points.

1.9.1 1.4.1 The simplest possible model: body mass index against disease progression

`load_diabetes` ships with scikit-learn, so there is no file to find. It holds 442 patients. Ten measurements were taken on each at baseline, and the target is a number describing how far the disease had progressed a year later: higher is worse.

One measurement only is used, column 2, body mass index. The features arrive already standardised, so the values sit near zero rather than near 25. That changes nothing about the method.

One line builds the model, one line fits it, one line scores it.

> One measurement, one straight line, a whole human body. Out of a possible 1.0, write down the R^2 you expect before you run the next two cells.

1.10 Quick Recall

- The **linear model** is $\hat{y} = mx + c$: m is the change in $\hat{y}$ per unit of x, c is $\hat{y}$ at $x = 0$.
- Fitting means choosing m and c to make the squared errors small.
- R^2 is the fraction of the variance in y the model explains. 0 means no better than the mean.
- **Bias** is error from a model too rigid, **variance** error from a model too eager, **irreducible noise** is in the data.
- The **loss** is minimised during training. The **metric** is reported afterwards. They are different numbers.

[20]:
```
from sklearn.datasets import load_diabetes

diabetes = load_diabetes()
print("shape:", diabetes.data.shape)
print("feature names:", list(diabetes.feature_names))
print("target: disease progression, from", diabetes.target.min(), "to",␣
 ↪diabetes.target.max())
```

```
shape: (442, 10)
feature names: ['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6']
target: disease progression, from 25.0 to 346.0
```

[21]:
```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

X = diabetes.data[:, [2]]
y = diabetes.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

linreg = LinearRegression().fit(X_train, y_train)
print(f"slope = {linreg.coef_[0]:.2f}   intercept = {linreg.intercept_:.2f}")
print(f"R^2 on the held-back test set = {linreg.score(X_test, y_test):.3f}")
print()
print(f"Read the slope as a sentence: one standard-deviation increase in body␣
 ↪mass index")
print(f"goes with {linreg.coef_[0]:.0f} more units of disease progression a␣
 ↪year later.")
print(f"Read the intercept as: a patient of average body mass index scores␣
 ↪{linreg.intercept_:.0f}.")
```

```
slope = 998.58   intercept = 152.00
R^2 on the held-back test set = 0.233
```

```
Read the slope as a sentence: one standard-deviation increase in body mass index
goes with 999 more units of disease progression a year later.
Read the intercept as: a patient of average body mass index scores 152.
```

1.10.1 1.4.2 R-squared, the regression score

For regression, the default score is **R-squared** (R^2). Read it in one sentence:

> R^2 is the **fraction of the wobble in y that the line explains**.

- $R^2 = 1$ means a perfect fit (the line explains everything).
- $R^2 = 0$ means the line is no better than just guessing the average y.
- $R^2 < 0$ means the line is *worse* than guessing the average (something is very wrong).

It is always between 0 and 1 for a sensible fit on the test set, and bigger is better.

Figure 1.10 fits a straight line to the diabetes data and reports the R-squared on the test set. The scatter around the line is what R-squared fails to capture.

[22]:
```
fig = plt.figure(figsize=(6.5, 4.5))
plt.scatter(X_test, y_test, c="#55A868", s=25, alpha=0.7, label="test data")
xs = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
plt.plot(xs, linreg.predict(xs), c="#C44E52", lw=2, label="fitted line")
plt.xlabel("body mass index (standardised units)"); plt.ylabel("disease␣
 ↪progression after one year")
plt.legend()
fig.text(0.5, 0.01, f"Figure 1.10. Linear regression fit (R^2 = {linreg.
 ↪score(X_test, y_test):.2f})", ha="center", fontsize=10)
plt.show()
```

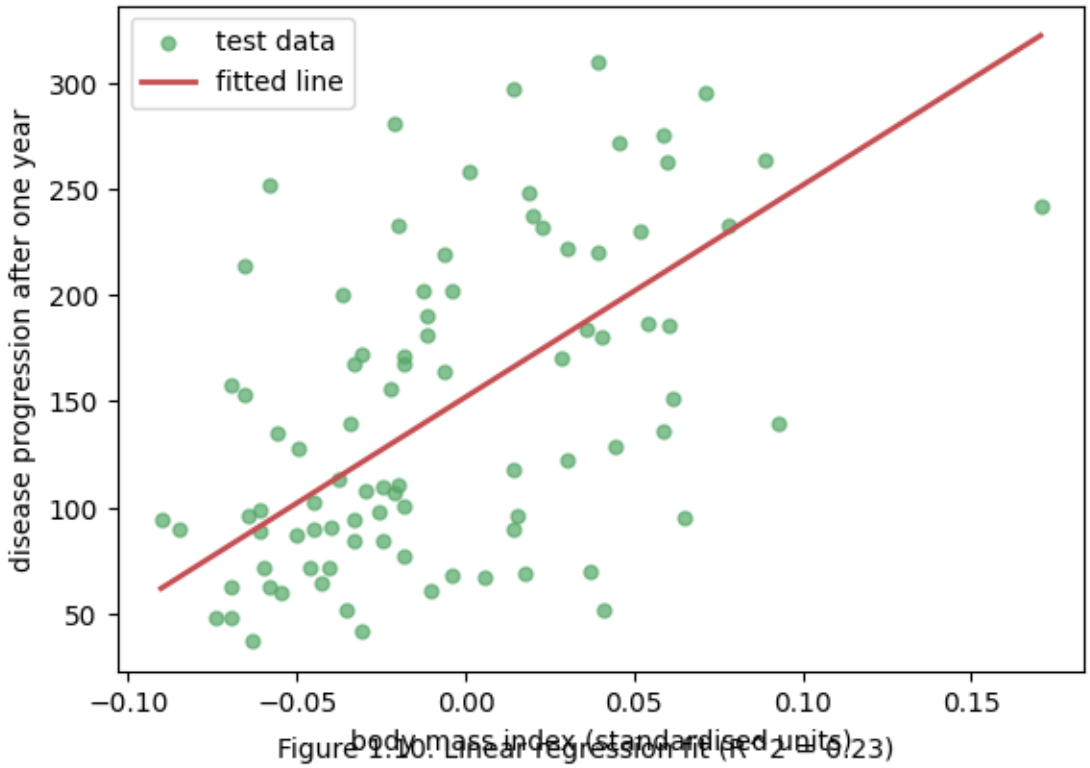


Figure 1.10: Linear regression fit ($R^2 = 0.23$)

1.10.2 1.4.3 Holding data back, and why

- **Training set**, the rows the model fits on.
- **Test set**, sealed until the end and touched once.
- **Cross-validation**, five folds carved out of the training set, each held out in turn to tune `k` or a polynomial degree. No separate validation set is cut off, so no data is wasted when you have little of it.

Figure 1.12 is the split itself: fit on 80%, judge once on 20%, tune `k` by cross-validation inside the training portion.

> **The golden rule of machine learning.** The test set cannot influence training in any way. Not the fit, not the choice of `k`, not the preprocessing. Touch it once, at the very end, to answer one question: does this work on data nobody has seen? Peek at it repeatedly and it stops being a test set, because you start tuning your choices to it and its score drifts up for reasons that have nothing to do with the model.

[23]:
```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(10, 2.3))
train_n, test_n = 80, 20
ax.barh(0, train_n / 100, height=0.6, color="#4C72B0", edgecolor="white")
```

```
ax.barh(0, test_n / 100, left=train_n / 100, height=0.6, color="#C44E52",␣
  ↪edgecolor="white")
ax.axvline(train_n / 100, color="white", lw=2.5)
for fold in range(1, 5):
    ax.axvline(train_n * fold / 5 / 100, color="white", lw=1.0, alpha=0.5)
ax.text(train_n / 200, 0, "train\n80%\n(5 CV folds)", ha="center", va="center",␣
  ↪color="white", fontsize=10, fontweight="bold")
ax.text((train_n + test_n / 2) / 100, 0, "test\n20%", ha="center", va="center",␣
  ↪color="white", fontsize=11, fontweight="bold")
ax.set_xlim(0, 1); ax.set_ylim(-0.45, 0.45)
ax.set_yticks([]); ax.set_xticks([])
fig.text(0.5, 0.01, "Figure 1.12. The train/test split: fit on 80%, judge once␣
  ↪on 20% (k tuned by CV inside train)", ha="center", fontsize=10)
plt.tight_layout(); plt.show()
```

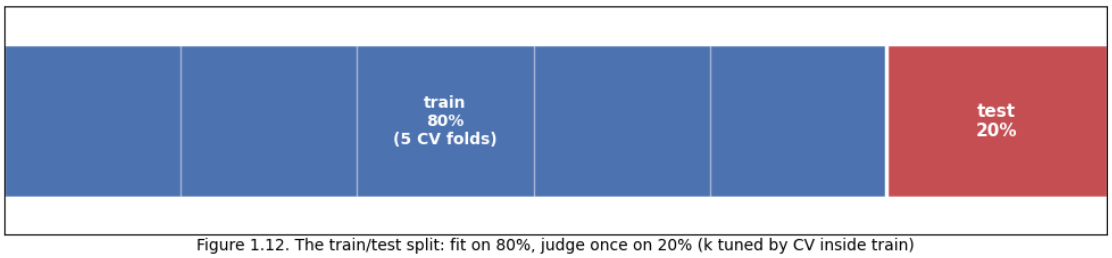


Figure 1.12. The train/test split: fit on 80%, judge once on 20% (k tuned by CV inside train)

1.10.3 1.4.3a Cross-validation: scoring without wasting data

A single train/test split scores the model on 20% of the data and throws the rest away on testing. With 442 patients that is wasteful, and the score depends on which 20% happened to land in the test set.

k-fold cross-validation splits the data into k equal pieces called folds. The model trains on $k-1$ folds and scores on the one left out, then repeats so every fold takes a turn being the test set. Nothing is wasted, and the k scores show how stable the answer is.

The score reported is the **mean plus or minus the standard deviation** across the k folds. A tight spread means the model is consistent; a wide spread means it depends heavily on which data it saw. The 2026 exam asks this in Question 6b.

[24]:
```
# 5-fold cross-validation on the moons data from Figure 1.9
from sklearn.model_selection import cross_val_score, KFold

kf = KFold(n_splits=5, shuffle=True, random_state=42)
knn_cv = KNeighborsClassifier(n_neighbors=5)
scores = cross_val_score(knn_cv, Xm, ym, cv=kf)

print("per-fold accuracy:", np.round(scores, 3))
```

```
print(f"5-fold CV = {scores.mean():.3f} ± {scores.std():.3f}")
```

```
per-fold accuracy: [0.917 0.9   0.9   0.8   0.917]
5-fold CV = 0.887 ± 0.044
```

[25]:
```
# Sweep k (number of neighbours) and read off cross-validation
records = []
k_values = [1, 3, 5, 15]
for k in k_values:
    s = cross_val_score(KNeighborsClassifier(n_neighbors=k), Xm, ym, cv=kf)
    for acc in s:
        records.append({"k": k, "accuracy": acc})
    print(f"k={k:2d}: {s.mean():.3f} ± {s.std():.3f}   per-fold {np.round(s,␣
 ↪3)}")

cv_df = pd.DataFrame(records)

fig, ax = plt.subplots(figsize=(6.2, 3.8))
sns.boxplot(data=cv_df, x="k", y="accuracy", ax=ax, color="#4C72B0", width=0.5,␣
 ↪fliersize=0)
sns.stripplot(data=cv_df, x="k", y="accuracy", ax=ax, color="black", size=4,␣
 ↪jitter=0.12)
ax.set_xlabel("k (number of neighbours)"); ax.set_ylabel("cross-validation␣
 ↪accuracy")
ax.set_title("5-fold CV accuracy against k")
fig.text(0.5, 0.01, "Figure 1.11. Cross-validation picks k without wasting␣
 ↪data", ha="center", fontsize=9)
plt.tight_layout(rect=[0, 0.04, 1, 1]); plt.show()
```

```
k= 1: 0.870 ± 0.027   per-fold [0.867 0.867 0.917 0.833 0.867]
k= 3: 0.883 ± 0.038   per-fold [0.9   0.883 0.883 0.817 0.933]
k= 5: 0.887 ± 0.044   per-fold [0.917 0.9   0.9   0.8   0.917]
k=15: 0.883 ± 0.035   per-fold [0.9   0.917 0.883 0.817 0.9  ]
```

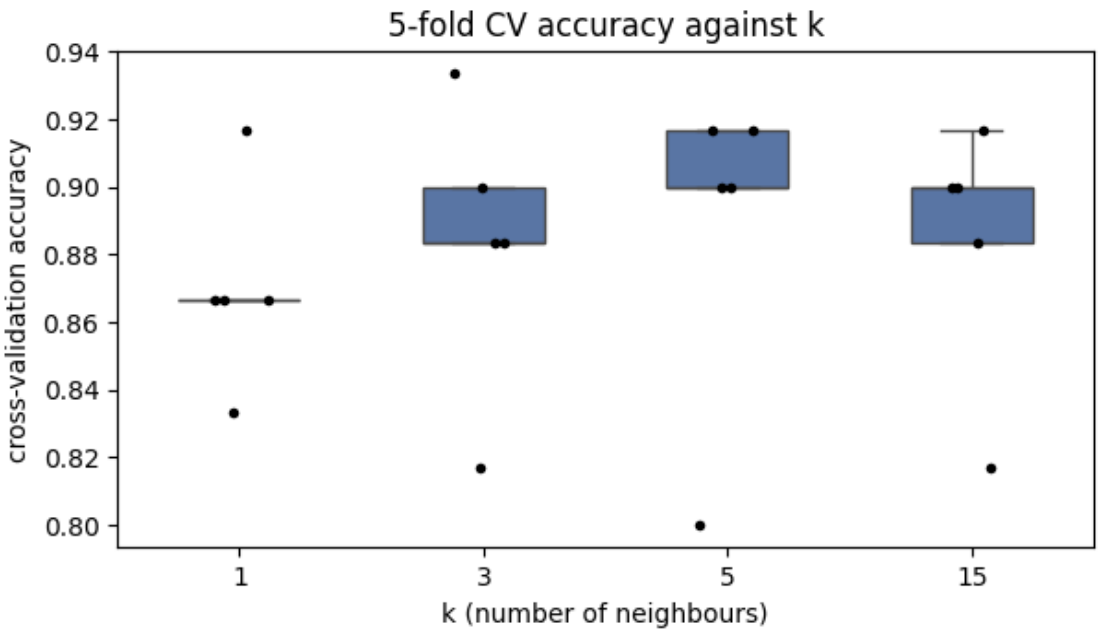


Figure 1.11. Cross-validation picks k without wasting data

1.10.4 1.4.3b Best practices: choosing k and repeating

The choice of k balances speed against stability. Five folds are used throughout this course for speed: each holds out a fifth of the data and retrains on the other four-fifths. Ten folds is the standard in the literature. Each of the ten fits is tested on a held-out tenth, and the mean is slightly steadier at a higher cost.

A single k-fold run can still mislead when the score is noisy. The noise has two sources. The split itself is random, so an unlucky fold boundary shifts the score. Some algorithms are also stochastic: a neural net or a random forest trained twice on the same data returns two different scores. When either source is large, repeat the whole k-fold run. Common choices are five-fold with two repeats, or ten repeats. The mean and standard deviation are then taken across all runs.

[26]:
```
# Repeated 5-fold CV: 5 folds, 2 repeats, on the same k-NN
from sklearn.model_selection import RepeatedKFold

rkf = RepeatedKFold(n_splits=5, n_repeats=2, random_state=42)
scores_rep = cross_val_score(KNeighborsClassifier(n_neighbors=5), Xm, ym,␣
 ↪cv=rkf)

print(f"5-fold x 1  = {scores.mean():.3f} +/- {scores.std():.3f}  ␣
 ↪({len(scores)} folds)")
print(f"5-fold x 2  = {scores_rep.mean():.3f} +/- {scores_rep.std():.3f}  ␣
 ↪({len(scores_rep)} runs)")
```

```
5-fold x 1  = 0.887 +/- 0.044   (5 folds)
5-fold x 2  = 0.890 +/- 0.036   (10 runs)
```

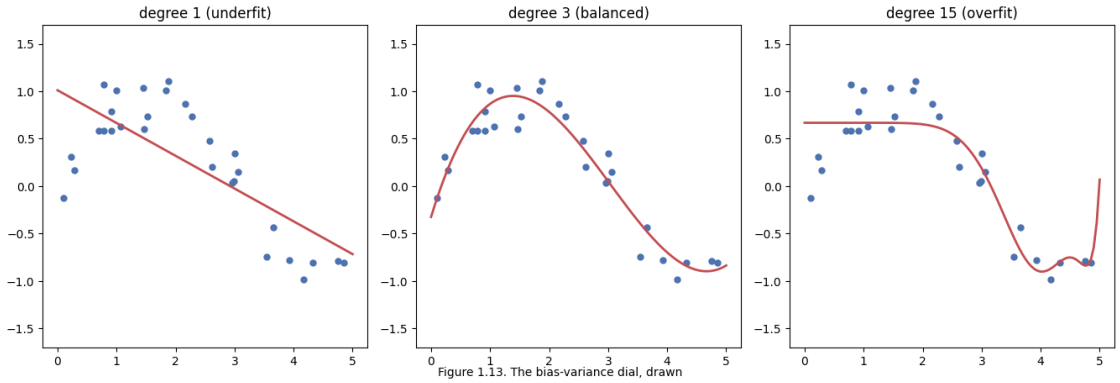
1.10.5 1.4.4 The bias-variance dial, drawn

The same dial from k-NN reappears here, now as model complexity. A straight line is a **low-complexity** model that cannot wiggle much. A high-degree **polynomial** can wiggle a great deal. **Figure 1.13** turns one dial through three settings on the same noisy data.

[27]:
```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline

rng = np.random.RandomState(42)
xs = np.sort(rng.uniform(0, 5, 30))
ys = np.sin(xs) + rng.normal(0, 0.2, 30)
Xp = xs.reshape(-1, 1)

fig, axes = plt.subplots(1, 3, figsize=(13.5, 4.6))
for ax, deg, title in zip(axes, [1, 3, 15],
                          ["degree 1 (underfit)", "degree 3 (balanced)", "degree␣
 ↪15 (overfit)"]):
    model = make_pipeline(PolynomialFeatures(deg), LinearRegression()).fit(Xp,␣
 ↪ys)
    ax.scatter(xs, ys, c="#4C72B0", s=25)
    xs_line = np.linspace(0, 5, 100).reshape(-1, 1)
    ax.plot(xs_line, model.predict(xs_line), c="#C44E52", lw=2)
    ax.set_ylim(-1.7, 1.7); ax.set_title(title)
plt.subplots_adjust(top=0.85)
fig.text(0.5, 0.01, "Figure 1.13. The bias-variance dial, drawn", ha="center",␣
 ↪fontsize=10)
plt.tight_layout(); plt.show()
```



That underfitting and overfitting are failures in opposite directions, not degrees of the same failure. The degree-1 line ignores structure that is really there. The degree-15 curve invents structure

that is not. Both score badly on held-out data, and only the middle panel would give a sensible prediction at a new x.

The degree-15 curve is the regression twin of the flag-everything spam model from Section 1.2.3: both chase a perfect training score by memorising, and both fail on new data. The degree-1 line is the always-not-spam model: too rigid to follow the truth. The degree-3 curve is the clever one, the only panel that knows the truth is a sine wave.

Why a degree-d polynomial is still a linear model

A degree-d polynomial turns one input x into $d+1$ coordinates: $x, x^2, \dots, x^d$. Lifting the input this way lets a straight line in the lifted space bend when drawn back in the original one. The model is still linear in its parameters $\theta_0, \theta_1, \dots, \theta_d$, which is why `LinearRegression` fits it.

The lift is also where the variance in the degree-15 panel comes from. Each new power adds a free knob the fit can turn, and with 15 knobs the curve has enough freedom to chase the noise in one training sample. Refit on a different sample and those knobs turn differently, so the curve shifts. That sensitivity to the exact training points is variance, and it is the same reason too many features hurt later chapters.

1.10.6 1.4.4a Bias, variance and the noise nobody can remove

- **Bias**: model too rigid to follow the truth. The degree-1 line is straight, the truth curves.
- **Variance**: model follows the training sample too closely. The degree-15 curve shifts on every refit.
- **Irreducible noise**: scatter in the data itself, not the model. The grey band a good model reaches but does not fit inside.

Figure 1.14. Bias, variance and irreducible noise. Total error has three sources. Bias is the gap between the rigid degree-1 line and the true curve; variance is how much the degree-15 curve shifts on a refit; irreducible noise is the grey band the data itself carries. A model whose error reaches the band has extracted everything; fitting inside it means fitting the noise.

1.10.7 1.4.5 Loss against metric, two different numbers

Table 1.2. Loss against metric: two different numbers for two different jobs

	Loss	Metric
Who reads it	the optimiser	a human
Must be	smooth, so training has a slope to follow	meaningful, nothing else
Example	mean squared error	accuracy, R^2

A model is trained on one number and judged on another. When somebody says a model "improved", ask which of the two moved.

Why neither number can do the other's job

Two numbers get thrown around loosely and they are not the same thing.

The **loss** is what the model minimises while it trains. It has to be smooth, because training adjusts the parameters in the direction the loss slopes downhill, and a non-smooth loss gives no slope to follow. Squared error is the usual choice for regression.

The **metric** is what you grade the finished model on, and it only has to be meaningful to a human. Accuracy counts correct answers. R^2 reports the fraction of the wobble explained.

Accuracy makes a poor loss, because nudging a weight slightly almost never flips a prediction, so the slope is zero nearly everywhere and training has nothing to follow. Squared error makes a poor report to a doctor, because nobody knows what a squared progression score means.

> **Golden thread.** Section 1.5 names the procedure that follows that downhill slope, and makes "roll down the loss" literal.
>
> One of these two numbers is better when it is small and the other is better when it is large. Predict which of the two the notebook reports to a human, and which one training actually minimised, before you run the cell.

[28]:
```
from sklearn.metrics import r2_score

preds = linreg.predict(X_test)
loss_value = ((preds - y_test) ** 2).mean()
metric_value = r2_score(y_test, preds)

print(f"loss   (mean squared error, what training minimised) = {loss_value:.
  ↪2f}")
print(f"metric (R-squared, what we report)                   = {metric_value:.
  ↪3f}")
print()
print("same model, same predictions, two numbers that move in opposite␣
  ↪directions")
```

```
loss   (mean squared error, what training minimised) = 4061.83
metric (R-squared, what we report)                   = 0.233

same model, same predictions, two numbers that move in opposite directions
```

1.10.8 Quick check, Section 1.4

1. A fit returns $m = 0$ and $c = \bar{y}$. What R^2 does it score on its training data?
2. Degree-15 polynomial, training error near zero, test error large. Bias or variance?
3. You collect ten times more data. Which of the three error contributions does that reduce, and which does it not touch?

Answers

1. Zero. Predicting the mean every time explains none of the variance, which is what $R^2 = 0$ means.
2. Variance. It has memorised the training sample rather than the relationship.
3. More data reduces variance. Bias stays, because the model is still the wrong shape, and irreducible noise stays because it belongs to the data.

1.11 1.5. Gradient descent, the golden thread

At the start we established that reducing the error during each iteration known as a step is the core of machine learning. The model, known as the Hypothesis, in linear regression is the line $\hat{y} = mx + c$. The two numbers m and c are its **parameters** (not tunable hyperparameters), the things the model *learns* from the data.

Those two numbers can be found directly by the **ordinary least-squares** formula, which solves for m and c in one shot (this is what `sklearn.linear_model.LinearRegression` did in Section 1.4). They can also be found by **gradient descent**, and gradient descent is the idea that reappears in every chapter after this one.

Picture a ball on the inside of a bowl. The height of the bowl is how wrong the model currently is. Let the ball go and it rolls downhill until it stops at the bottom. Gradient descent is that sentence written in symbols.

The loss here is the simplest possible bowl: $J(\theta) = \frac{1}{2}\theta^2$. The symbol θ is a single parameter the model is learning; on the real line it would stand in for m or for c, but here there is no data and just one knob, so the bowl is a clean curve. The bottom sits at $\theta = 0$, where the model is least wrong.

Why this shape, and why the $\frac{1}{2}$? The slope of the bowl at any θ is the derivative $\frac{dJ}{d\theta}$. Differentiate $\frac{1}{2}\theta^2$ with the power rule and the $\frac{1}{2}$ cancels the 2 the rule brings down, leaving a slope of exactly θ. So on this bowl the gradient is just θ itself: far from 0 the slope is steep, at $\theta = 0$ the slope is 0 and the ball stops. The $\frac{1}{2}$ is there only so the derivative comes out this clean. Nothing in the answer depends on a hidden number.

> **Theta is where you are; slope is how steep it is there.** Two different jobs — and on $\frac{1}{2}\theta^2$ the power rule makes the two numbers identical, which is exactly why this is the teaching bowl. Drop the $\frac{1}{2}$ and the slope becomes 2θ; on the real MSE cost the slope is $\frac{1}{m}\sum 2(y_i - \hat{y}_i)(-x_i)$, a sum over the data, not θ at all. The update $\theta \leftarrow \theta - \alpha \cdot \text{slope}$ is the same in every case; only the slope's formula changes. Click the reveal just below to see the cost formula that this slope came from: the cost J is the **height** of the bowl, and differentiating it is what produced the slope above.

Start the ball at $\theta = 4$ with learning rate $\alpha = 0.5$. The update $\theta \leftarrow \theta - \alpha\,\theta$ becomes *take half of where you are*, and the steps are visible: $4 \to 2 \to 1 \to 0.5 \to 0$. The slope and θ share a value here only because the gradient of $\frac{1}{2}\theta^2$ is θ; the quantity that moves is the step $\alpha\,\theta$, which halves each time.

This toy bowl has one parameter and no data, which is why its shape is so clean. A real regression line has two parameters, m and c, and its bowl is built from the data: the **mean squared error**, the average squared gap between each guess $\hat{y}_i$ and the truth y_i. Gradient descent rolls the ball down that bowl by the same rule, only the slope is now a vector with one entry per parameter instead of a single number.

Show the mean squared error cost for a line

$$J = \text{MSE} = \frac{1}{m}\sum_{i=1}^{m}\left(y_i - \hat{y}_i\right)^2, \qquad \hat{y}_i = m\,x_i + c$$

The m under the sum is the number of training samples (see the notation section). Each term

$(y_i - \hat{y}_i)$ is the **residual**, how far the line's guess misses the true y_i; squaring makes every miss positive so they cannot cancel, and averaging turns the total into a per-point cost. The same shape as $\frac{1}{2}\theta^2$ sits inside: square the error, average it, roll downhill. The cell in Section 1.4.5 already printed this loss for the diabetes line as the number training minimised, next to the R^2 reported to a human.

Show the gradient-descent update rule

$$\theta \leftarrow \theta - \alpha \cdot \text{slope}$$

Read the arrow $\leftarrow$ as **becomes**: the new θ becomes the old θ minus a nudge. θ is the parameter the model is learning, here standing in for m or c. α is the **learning rate**, the step size. The slope says which way is down and how steep, and the minus sign turns slope into downhill, which is why the update subtracts it.

Golden thread. The line $\hat{y} = mx + c$ and the neural networks in `ch3` are different models, rolled down the same bowl by this one rule.

1.12 Quick Recall

- **Gradient descent** takes repeated steps downhill on the loss: $\theta \leftarrow \theta - \alpha \cdot \text{slope}$.
- The **slope** points uphill, which is why the update subtracts it.
- The **learning rate** α scales the step. Too small crawls, too large overshoots and diverges.
- Every model in `ch3`, up to a convolutional network, is trained by this one loop.

[31]:
```
theta = 4.0
alpha = 0.5
history = [theta]
for step in range(14):
    dJ = theta            # slope of J = 0.5 * theta**2 is just theta
    theta = theta - alpha * dJ
    history.append(theta)

xs_bowl = np.linspace(-6.0, 6.0, 300)

fig, ax = plt.subplots(figsize=(7.5, 5.4))
ax.plot(xs_bowl, 0.5 * xs_bowl ** 2, c="#4C72B0", lw=2, label="loss J")
ax.set_xlim(-6, 6); ax.set_ylim(-1, 10)
ax.set_xlabel("theta (the parameter being learned)"); ax.set_ylabel("J (how␣
 ↪wrong the model is)")
ax.legend(loc="upper right")

trail_pts, = ax.plot([], [], "o-", c="#C44E52", ms=5, lw=1, alpha=0.5)
ball, = ax.plot([], [], "o", c="#C44E52", ms=13)
tangent, = ax.plot([], [], c="#DD8452", lw=2.5)
readout = ax.text(0.02, 0.96, "", transform=ax.transAxes, va="top", fontsize=9,
                  bbox=dict(boxstyle="round,pad=0.4", fc="white", ec="grey",␣
 ↪alpha=0.9))
```

```
def update(i):
    t = history[i]
    j = 0.5 * t ** 2
    slope = t               # gradient of 0.5*theta**2 is theta
    seg = np.array([t - 1.8, t + 1.8])
    tangent.set_data(seg, j + slope * (seg - t))
    ball.set_data([t], [j])
    trail_pts.set_data(history[:i + 1], 0.5 * np.array(history[:i + 1]) ** 2)
    step_size = alpha * slope
    readout.set_text(
        f"theta = {t:+.3f}\n"
        f"J = {j:.3f}\n"
        f"step = alpha x slope = {alpha} x {t:.3f} = {step_size:.3f}\n"
        f"theta becomes {t:+.3f} - {step_size:.3f} = {t - step_size:+.3f}")
    ax.set_title(f"step {i}: the tangent flattens as the ball settles")
    return ball, tangent

fig.text(0.5, 0.02, "Figure 1.15. Gradient descent: ball, slope, step",␣
 ↪ha="center", fontsize=9)
plt.subplots_adjust(bottom=0.18)

anim_obj = anim.FuncAnimation(fig, update, frames=len(history), interval=900,
                              blit=False, repeat=True)
plt.close(fig)
_g = tempfile.NamedTemporaryFile(suffix=".gif", delete=False); _g.close()
anim_obj.save(_g.name, writer="pillow")
with open(_g.name, "rb") as _f: _d = _f.read()
os.unlink(_g.name)
HTML(f'<img src="{base64.b64encode(_d).decode()}">')
```

[31]: <IPython.core.display.HTML object>

That the slope is the whole instruction set. The tangent line drawn at the ball is just the slope made visible, so where the curve is steep the tangent is steep and the step is long; near the minimum the tangent flattens and the steps shrink by themselves. No rule tells the ball to slow down. Multiply the same rule by thousands of parameters and you have trained a neural network.

1.12.1 Quick quiz

1. A model gets $R^2 = -0.3$ on the test set. What does the negative sign tell you, and what would be an even simpler "model" that beats it?
2. Why is scoring done on a **held-out test set** rather than the training data? Give the one-word failure mode the held-out set avoids.
3. In the update rule above, what happens if the learning rate α is far too large? What if it is far too small?
4. A degree-15 polynomial scores better on the *training* set than a degree-3 polynomial, but worse on the *test* set. Name what is happening, in bias-variance terms.

Answers (click reveal)

1. The model is **worse than guessing the mean** of y. An even simpler model that beats it: predict the average y for everything (a flat horizontal line), which gives $R^2 \approx 0$.
2. **Overfitting**. If scoring is done on training data, a model that memorises the training set (noise included) looks artificially good. The held-out set measures generalisation.
3. Too large: the steps overshoot the bottom and **diverge** (theta bounces out of the bowl). Too small: each step is tiny, so the model **converges very slowly** (or never finishes in practice).
4. **Overfitting** - high variance. The degree-15 polynomial has memorised the training noise, so it scores well on training but badly on new (test) data. The degree-3 generalises better.

1.12.2 Quick check, Section 1.5

1. Why does the update subtract the gradient rather than add it?
2. Your loss rises every step instead of falling. What is the first thing to change?
3. At a minimum, what is the gradient, and what does the update do there?

Answers

1. The gradient points in the direction of steepest increase, and the loss is to be decreased.
2. The learning rate, downwards. Steps large enough to jump past the minimum land higher up the other side each time.
3. Zero, so the update leaves the parameter where it is. That is what having converged means.

1.13 Visual appendix

Table 1.3. Visual appendix: every figure in this chapter and what it demonstrates

Every figure in this chapter, and the one thing each is there to show.

Figure	Section	What it demonstrates
1.1	opener	A line finding m and c from 442 patients, error falling over 14 steps
1.2, 1.3	1.2.1	The four confusion counts, and the four scores built from them
1.4, 1.5	1.3.1	Eleven supporters on a coordinate plane, then a newcomer joining them
1.6	1.3.1	A 3-NN vote made visible, one line per neighbour
1.7, 1.8	1.3.2	The boundary straightening as `k` grows on two clean blobs
1.9	1.3.2a	The same dial on interlocking crescents, where it costs accuracy
1.10	1.4.2	The regression fit to the diabetes data
1.11	1.4.3a	Cross-validation picking k without wasting data

Figure	Section	What it demonstrates
1.12	1.4.3	The train/test split, fit on 80% and judged once on 20%
1.13	1.4.4	Degree 1, 3 and 15 on the same points: underfit, fit, overfit
1.14	1.4.4a	Bias, variance and the irreducible noise band, named separately
1.15	1.5	Gradient descent on one parameter, steps shrinking near the minimum

Re-read any row you cannot answer from memory, then look at the figure again.

1.14 When it breaks

`ValueError: Expected 2D array, got 1D array instead.` scikit-learn wants `X` with two dimensions, rows by columns, even when there is one column. `data[:, 2]` gives a flat 1D array and fails; `data[:, [2]]` keeps the column and works. The same distinction is `df[[5]]` against `df[5]` in Pandas. If you are stuck with a flat array, `X.reshape(-1, 1)` fixes it.

A suspiciously high score. Check what you passed to `.score`. Scoring on `X_train` measures memory, not learning. The number you report always comes from data the model was never fitted on.

`ValueError: Found input variables with inconsistent numbers of samples.` `X` and `y` have different lengths. Usually one of them was filtered and the other was not. Print `X.shape` and `y.shape` before every split.

A negative R^2. Not a bug. It means the model does worse than always guessing the mean, which is worth knowing rather than hiding.

1.15 1.6. Where this goes next

You now have the whole traditional-ML scoring pipeline. A way to **grade** guesses, the confusion matrix and the four metrics built from it. Two first learners, **k-NN** which classifies by neighbour vote and **linear regression** which fits a line and reports R^2. And the **gradient-descent update rule**, the single line of maths that rolls a model downhill.

`ch2` picks up exactly here.

- **Logistic regression**, because you usually want a probability rather than a bare label, so the line gets squashed through a **sigmoid**. The same gradient rule fits it.
- **Preprocessing**, because k-NN broke the moment two features were measured in different units.
- **PCA**, squashing many features down to two so they can be plotted.
- **Trees and forests**, a completely different kind of model built from nested if/then rules.

The golden thread continues. Every one of those is another model rolled down the same bowl.